Htw. 01. Unit 2.

Polymorphism.

Three pillars of OO: Encapsulation, Inheritance (specification), Polymorphism (abstraction).

Definition: Greek for "many forms". Related objects can be treated as the same kind of object in client code, and at runtime object does the correct task.
(Polymorphism ≠ inheritance.)

Chess example.

* Bad client code: if (piece instanceof Pawn) {...} else if (piece instance of Rook) {...}

  - It works, but it's maintance nightmare! adding a new piece means editing it, error-prone, and verbose.

  - We ask objects about themselves and then tell them how to act, instead of just telling them to act.

* Pin in 3 steps:

  1. Establish a common interface: abstract class chessPiece { abstract Location[] getPossibleMoves(); }

  2. Implement it in each subclass (Pawn, Rook) with @Override

  3. Depend on the superclass in the client: for (chessPiece p: pieces) p.getPossibleMoves();

Consequences.

- Client code is shorter and clearer. (All code is client code of something, so code accordingly.)

- But we don't know what the client code will do until runtime, because it depends on the actual subclass, we trade one complexity for another.

* Extensibility improves: new behavior goes in new subclass and the client code stays unchanged.

Notas

Polymorphism in Java.

Generalization class: FarmAnimal. Specialization class: chicken.

ArrayList <FarmAnimal> farmAnimals = new ArrayList<>();

ArrayList <chicken> chickens = new ArrayList();

farmAnimals.add(chicken); // OK (a chicken is a farmAnimal)
farmAnimals.add(farmAnimal); // OK
chickens.add(chicken); // OK
chickens.add(farmAnimal); // Error (a FarmAnimal is not necessary a chicken)

* Overriding: new definition of an existing method in a sub class (inheritance). e.g. to strings() or computeFoodCost() in Chicken overriding Farm Animal.

* overloading: several definitions of methods with the same name in the same class (different parameters).

Abstraction

Definitions.

* Generality: summarizing or generalizing something to focus on the ideas most relevant to a purpose.
* Process (verb): bringing details to the fore front while suppressing others. In Programming, it is creating an interface for a component that exposes the details needed to work with it and hides the rest.
* Product (noun): a description or definition that leaves out unnecessary details. A class definition is an abstraction.
* Gutting: the essence is to preserve information relevant in a given context and forget what is irrelevant in that context
* Public members expose the interesting details. Private members and method implementations have details. users don't need. Irrelevant details are left out entirely.
* context matters: a chicken class (id, breed, burOnly is Molting, get Age in months) for a farm information system would be very different in a chicken-coop simulator. Likewise, "Person" differs between a foot ball tournament system and a airline reservation system.

Abstraction in the chess example.

* Abstraction = considering only the relevant parts of an object and ignoring the others.

* Chess Piece is more abstract than Pawn (Pawn details: just location and set PossibleMoves()).
* Class activity (bar simulator vs dinosaur game): find the similarities (moving on a 2D grid, picking up items, user-controlled objects), put them in an abstract layer, and delegate the differences to concrete subclasses.
The abstract layer can't stand on its own, which is why it is abstract.

Principle: Each component has an explicit, clear declaration of its accessible features no more and no less than what other components need.
Benefit: understandability, testability, maintainability, reusability, and better modularity (it reveals poor localization of design decisions, unnecessary coupling and low cohesion).

Two common problems.
* Leaky abstraction: other components rely on details not explicitly started. Good encapsulation often solves it.

* Over-abstraction: too much is "forgotten", which gives users insufficient control or access to information.

Adherence criteria.

1. Meaningful identifiers: precise names, not too broad or narrow, classes and variable are nouns, methods are verbs

2. Context-aware identifiers: don't repeat information the context already gives (e.g., Person, getPersonName() * Person.getName()).

3. Explicit declaration, everything exposed must be part of the class definition or the documentation. Exposure can be explicit (public, protected) implicit (e.g. parameter validation) or accidental (e.g. returning a mutable internal object. Some constrains, like, method call order, can't be captured in the class itself

4. Sufficiency (completeness)

5. Concision (non-redundancy).

3. Interfaces and Abstract classes in Java.

abstract class: cannot be instaticated.
abstract method: the derived class must implement it.

interface: a reusable definition of some or all of a class's public interface. An interface can inherit from another interface, and a class implement an interface.

Keywords: abstract, interface, implements, @Override.

What compile and what doesn't

```
IFarmAnimal farmAnimal;          // OK (declaring a reference)

FarmAnimal = new FarmAnimal();   // ERROR (can't instaticate an
FarmAnimal = new Chicken();      //  interface)
                                 // OK
Collection<Person> persons;      // OK
persons = new Collection<>();    // ERROR (interface)
persons = new ArrayList<>();     // OK (concrete class)

Set<Integer> integers;           // OK
integers = new set<>();          // ERROR (interface)
integers = new HashSet<>();      // OK
```